

AI Chat Application Development Process

Overview

The goal of this project was to build a full-stack AI chat application that allows users to interact with a generative AI model through a clean and responsive interface. In addition to meeting the functional requirements, I focused on writing maintainable code, protecting sensitive information, and creating an intuitive user experience.

Planning the Architecture

How

Before writing any code, I divided the application into two separate projects:

- A React frontend
- An Express backend

The frontend is responsible for rendering the user interface and managing user interactions, while the backend handles communication with the Hugging Face Inference API.

Why

Separating the frontend from the backend keeps the application organized and prevents sensitive API credentials from being exposed in the browser. It also creates a scalable architecture that can be expanded with authentication, databases, or additional APIs in the future.

Building the Frontend

How

I built the user interface using React with Vite and organized the application into reusable components such as:

- Header
- ChatBox
- Message
- PromptInput
- LoadingSpinner
- ErrorMessage

Each component has a single responsibility, making the application easier to understand, test, and maintain.

Why

Breaking the application into reusable components follows React best practices and reduces code duplication. It also makes future changes easier because updates to one component do not affect unrelated parts of the application.

Managing Application State

How

React's `useState` hook manages:

- Chat messages
- Loading state
- Error messages

When a user submits a prompt, the application immediately updates the interface, displays a loading indicator, and renders the AI response once it is received.

Why

Using React state allows the interface to update dynamically without reloading the page, creating a smoother and more responsive user experience.

Integrating the AI Model

How

The frontend sends user prompts to the Express backend using Axios. The backend validates the request and communicates with the Hugging Face Inference SDK. Once the AI generates a response, the backend returns it to the frontend, where it is displayed in the chat.

Why

Routing requests through the backend protects the API key and separates business logic from the user interface. This approach follows common full-stack development practices and improves application security.

Improving the User Experience

How

To make the application more user-friendly, I implemented:

- Loading indicators while waiting for AI responses
- Markdown rendering for formatted responses
- Persistent chat history using LocalStorage
- A "New Chat" button to reset the conversation
- Responsive layouts for desktop and mobile devices
- Keyboard submission using the Enter key

Why

These features make the application feel more polished and intuitive. Persistent chat history allows users to continue conversations after refreshing the page, while Markdown support improves the readability of AI-generated content, especially for lists and code snippets.

Error Handling

How

Validation was implemented on both the frontend and backend. The application checks for empty prompts, handles API failures, and catches unexpected server errors before displaying user-friendly error messages.

Why

Users should receive clear feedback when something goes wrong instead of the application failing silently or displaying technical errors. Proper error handling also makes the application easier to debug and maintain.

Testing

How

I added unit tests using Vitest and React Testing Library for several core components to verify that they render correctly and respond to user interaction as expected.

Why

Although this is a relatively small project, automated testing helps identify regressions during future development and demonstrates familiarity with modern frontend testing practices.

Challenges and Solutions

Hugging Face Authentication

Initially, the application returned authorization errors because the API token did not have the required permissions. I resolved this by generating a new token with the appropriate access level and validating requests using Postman before integrating them into the frontend.

Response Formatting

Some AI responses were difficult to read because they were returned as plain text. To improve readability, I integrated React Markdown with GitHub Flavored Markdown support so that headings, lists, links, and code blocks render correctly.

Response Length

During testing, longer AI responses were occasionally truncated. After investigating, I found that the issue was caused by the maximum token limit configured for the Hugging Face request. Increasing the token limit allowed the model to return complete responses.

Future Improvements

If I continued developing this project, I would add:

- User authentication
- Database-backed conversation history
- Streaming AI responses
- Syntax highlighting for code blocks
- Multiple AI model selection
- Docker support
- CI/CD automation

These enhancements would improve scalability and move the application closer to a production-ready solution.

Conclusion

This project demonstrates my ability to design and implement a full-stack web application using React, Express, and the Hugging Face Inference SDK. Throughout development, I focused on building a secure, maintainable, and responsive application while following modern development practices. In addition to meeting the required functionality, I incorporated several bonus features—including persistent chat history, Markdown rendering, and automated testing—to create a more complete user experience.